

Linux Basics & Installation

Complete Day 1 Notes — Embedded Linux Track

DAY 1 · FOUNDATIONS

Linux Theory & Kernel Architecture • Hands-On Installation Guide • Revision & Practice

Table of Contents

Part 1 — Linux Theory & Concepts

- 1.1 Linux History
- 1.2 Linux Kernel Architecture
- 1.3 Kernel Space vs User Space
- 1.4 Linux Kernel Features
- 1.5 Why Linux for Embedded Systems?

Part 2 — Hands-On Practice: Step-by-Step Guide

- 2.1 Install Ubuntu in VirtualBox
- 2.2 Install Ubuntu in WSL2 (Windows)
- 2.3 Create Directory Structure
- 2.4 Essential Linux Commands for Day 1

Part 3 — Revision Material

- 3.1 Linux Filesystem Hierarchy
- 3.2 Day 1 Quiz — Test Yourself
- 3.3 Day 1 Practical Assignment
- 3.4 Common Error Fixes
- 3.5 Day 1 Summary Notes & Completion Checklist

Part 1: Linux Theory & Concepts

1.1 Linux History

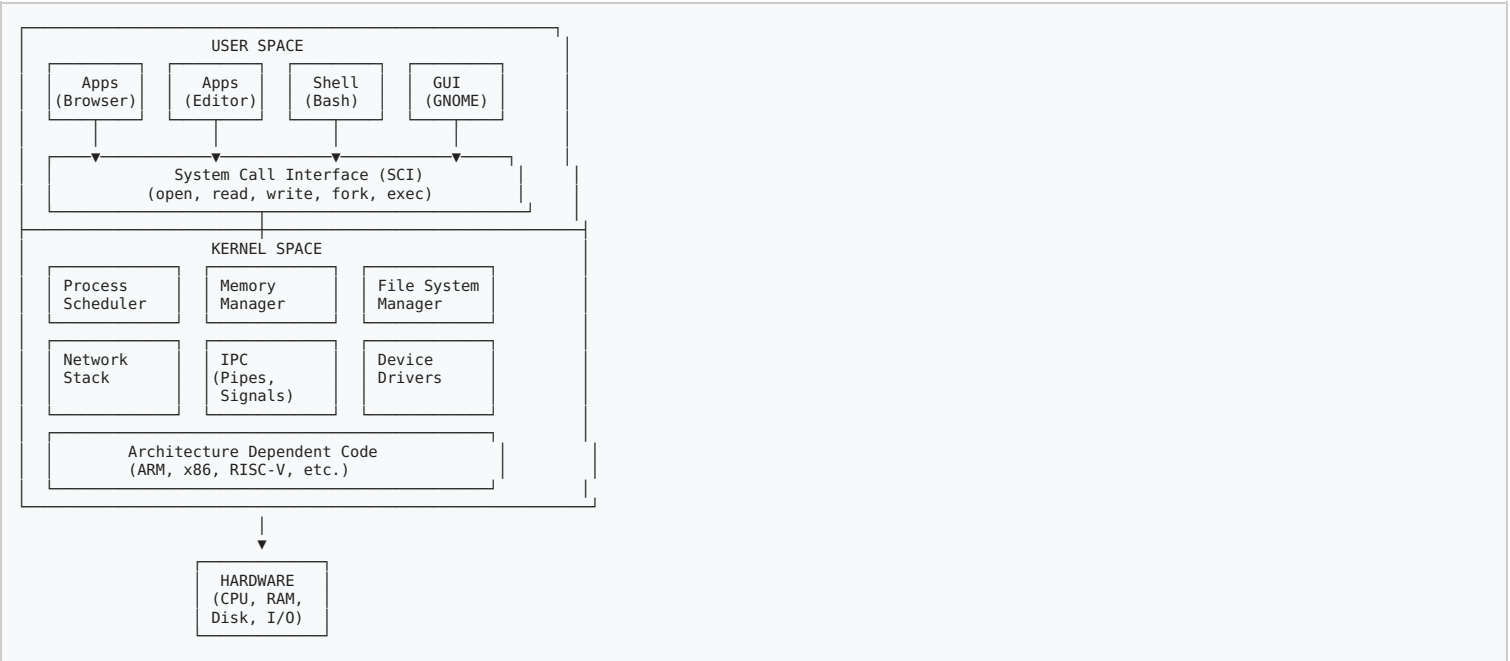
Year	Event
1969	Unix created by Ken Thompson & Dennis Ritchie at AT&T Bell Labs
1983	Richard Stallman starts GNU Project (Free Software Foundation)
1991	Linus Torvalds (Finnish student) releases Linux kernel v0.01
1992	Linux kernel v0.12 released under GNU General Public License (GPL)
1994	Linux v1.0.0 released with 176,250 lines of code
2024	Linux kernel has over 30+ million lines of code

Why "Linux"? = Linus + Unix (combination)

Key Points to Remember

- Linux is **NOT** an operating system itself — it's just the **KERNEL**
- The complete OS is called **GNU/Linux** (kernel + GNU tools + applications)
- Linux is Open Source — anyone can view, modify, and distribute code
- Over 95% of servers run Linux
- 90%+ of supercomputers run Linux
- Billions of Android phones run Linux kernel

1.2 Linux Kernel Architecture



1.3 Kernel Space vs User Space — Detailed Comparison

Feature	Kernel Space	User Space
Privilege Level	Ring 0 (Highest)	Ring 3 (Lowest)
Hardware Access	Direct access to all hardware	Must use System Calls
Memory Access	Full memory access	Virtual memory only
Crash Impact	System crashes (Kernel Panic)	Only that application crashes
Code Examples	Device drivers, scheduler, memory mgr	Browser, text editor, games
Debugging	Very difficult (needs kernel debugger)	Easy (GDB, print statements)

System Call Process Example

```
// When you write this in user space:
int fd = open("/tmp/file.txt", O_RDONLY);

// The kernel does this:
// 1. Trap into kernel mode (software interrupt)
// 2. Copy file path from user to kernel space
// 3. Check permissions
// 4. Access hardware (disk)
// 5. Return file descriptor to user space
```

1.4 Linux Kernel Features (Detailed List)

Feature	Description	Embedded Relevance
Monolithic Kernel	Single large binary with all services	Efficient for resource-constrained devices
Modular Design	Loadable Kernel Modules (LKMs)	Add drivers without reboot
Multi-User	Multiple users simultaneously	Industrial IoT applications
Multi-Tasking	Preemptive multitasking	Real-time responsive systems
Virtual Memory	Swap space, memory mapping	Efficient RAM usage
Symmetric Multi-Processing (SMP)	Multiple CPU cores supported	Performance scaling
POSIX Compliance	Standard API	Code portability
File Systems	Ext4, FAT, NTFS, NFS, etc.	Storage flexibility
Networking	TCP/IP stack, sockets	IoT connectivity
Security	SELinux, AppArmor, Capabilities	Device security
Device Drivers	Thousands of supported devices	Hardware compatibility
Real-Time Capabilities	PREEMPT_RT patch	Time-critical applications

1.5 Why Linux for Embedded Systems? — Complete Answer

10 Reasons with Detailed Explanation:

1. Open Source & Cost

- Zero licensing fees
- Freedom to modify source code
- No vendor lock-in
- Community support

2. Hardware Support

Supports over 30+ CPU architectures:

- ARM (Cortex-A, Cortex-M, Cortex-R)
- x86 / x86_64
- RISC-V
- MIPS
- PowerPC
- And many more

3. Customizability

- Can strip down kernel to just 1-2MB for tiny devices
- Add only needed features
- Example: Buildroot and Yocto Project create custom Linux distributions

4. Real-Time Capabilities

- PREEMPT_RT patch provides hard real-time
- Latency as low as 10-50 microseconds
- Used in: Medical devices, Automotive (ECUs), Industrial robots

5. Vast Ecosystem

- 50,000+ available packages
- Libraries, frameworks, drivers
- Python, C, C++, Rust support

6. Networking Stack

- Full TCP/IP stack built-in
- Supports: Ethernet, Wi-Fi, Bluetooth, CAN, Zigbee, LoRa
- Internet of Things ready

7. Development Tools

- GCC, GDB, Make, CMake
- Eclipse, VS Code, Qt Creator
- Cross-compilation toolchains

8. Security Features

- SELinux, AppArmor
- Secure boot support
- Encrypted filesystems
- Regular security updates

9. Long-Term Support (LTS)

- LTS kernels supported for 5-10 years
- Stable APIs for drivers
- Enterprise-level reliability

10. Community & Documentation

- Massive developer community
- Extensive documentation
- Quick bug fixes
- Forums, mailing lists, Stack Overflow

Comparison: Linux vs RTOS for Embedded

Aspect	Linux	RTOS (FreeRTOS, ThreadX)
Complexity	High	Low to Medium
Footprint	1MB - 1GB	2KB - 500KB
Features	Full OS	Minimal OS
Real-Time	Soft real-time (Hard with PREEMPT_RT)	Hard real-time
Networking	Full TCP/IP stack	Limited or add-on
Development	Easy (Linux tools)	Harder (proprietary tools)
Cost	Free	Often paid (commercial)
Use Cases	IoT, Automotive, Medical	Microcontrollers, Motor Control

Part 2: Hands-On Practice — Step-by-Step Guide

2.1 Install Ubuntu in VirtualBox (Complete Guide)

Step 1: Download Requirements

```
VirtualBox:    https://www.virtualbox.org/wiki/Downloads
Ubuntu ISO:    https://ubuntu.com/download/server
               OR
               https://ubuntu.com/download/desktop (if you want GUI)
```

Step 2: Install VirtualBox

```
# On Windows: Run installer as Administrator
# On Linux (Ubuntu/Debian):
sudo apt update
sudo apt install virtualbox virtualbox-ext-pack

# On macOS: Download .dmg and install
```

Step 3: Create Virtual Machine

1. Open VirtualBox → Click New
2. Set:
 - Name: Ubuntu-Embedded
 - Folder: C:\VirtualMachines\
 - ISO Image: Choose downloaded Ubuntu ISO
 - Type: Linux
 - Version: Ubuntu (64-bit)
3. Click Next

Step 4: Allocate Resources

```
Memory (RAM):  4096 MB (4GB) minimum
CPU Cores:      2 cores minimum
Hard Disk:      25-40 GB (dynamic allocation)
```

Step 5: Install Ubuntu

1. Start the VM
2. Select Install Ubuntu
3. Keyboard layout: US (or your preference)
4. Installation type: Erase disk and install Ubuntu (Inside VM only)
5. Fill user details:
 - Name: embedded-dev
 - Computer name: embedded-vm
 - Username: embed
 - Password: password123 (use strong for production)
6. Wait for installation (10-20 minutes)
7. Restart VM

Step 6: Post-Installation Setup

```
# Update system
sudo apt update && sudo apt upgrade -y

# Install essential tools
sudo apt install -y build-essential git vim curl wget \
gdb make cmake tree htop net-tools

# Install development tools for embedded
sudo apt install -y gcc-arm-linux-gnueabi \
gcc-arm-linux-gnueabi-hf \
gcc-aarch64-linux-gnu \
qemu qemu-user-static
```

2.2 Install Ubuntu in WSL2 (Windows Only, Complete Guide)

Step 1: Enable WSL (Windows 10/11)

```
# Open PowerShell as Administrator
# Enable Windows Subsystem for Linux
dism.exe /online /enable-feature /featurename:Microsoft-Windows-Subsystem-Linux /all /norestart

# Enable Virtual Machine Platform
dism.exe /online /enable-feature /featurename:VirtualMachinePlatform /all /norestart

# Restart your computer
```

Step 2: Set WSL2 as Default

```
# After restart, in PowerShell as Admin:
wsl --set-default-version 2
```

Step 3: Install Ubuntu

```
# Download and install Ubuntu
wsl --install -d Ubuntu-22.04
# OR
# From Microsoft Store: Search "Ubuntu 22.04 LTS" → Install
```

Step 4: First Time Setup

```
# After installation, launch Ubuntu from Start Menu
# Set up username and password (this is NOT your Windows login)
# Example:
Username: embed
Password: password123
```

Step 5: WSL2 Configuration for Development

```
# Update system
sudo apt update && sudo apt upgrade -y

# Install development tools
sudo apt install -y build-essential git vim curl wget \
    gdb make cmake tree htop net-tools

# Install cross-compilation tools
sudo apt install -y gcc-arm-linux-gnueabi \
    gcc-arm-linux-gnueabi \
    gcc-aarch64-linux-gnu \
    qemu-user-static

# Enable WSL2 features (create ~/.wslconfig in Windows)
# In Windows command prompt:
notepad "$env:USERPROFILE\.wslconfig"
# Add:
[network]
generateResolvConf = false

[filesystem]
caseSensitive = false

[wsl2]
memory=4GB
processors=2
localhostForwarding=true
```

WSL2 File System Locations

```
Windows:    C:\Users\<YourName>\AppData\Local\Packages\Canonical...
Linux:      \\wsl$\Ubuntu-22.04\home\embed\

# Access Linux files from Windows Explorer:
\\wsl.localhost\Ubuntu-22.04\home\embed
```

2.3 Create Directory Structure — Detailed Commands

```
# First, verify your current location
pwd
# Output: /home/embed

# Create the main project directory
mkdir -p ~/embedded_projects

# Create subdirectories one by one
```

```
mkdir ~/embedded_projects/c_programs
mkdir ~/embedded_projects/linux_scripts
mkdir ~/embedded_projects/projects

# OR all at once:
mkdir -p ~/embedded_projects/{c_programs,linux_scripts,projects}

# Verify structure
tree ~/embedded_projects/
# Output:
# /home/embed/embedded_projects/
# |— c_programs
# |— linux_scripts
# |— projects

# Check permissions
ls -la ~/embedded_projects/
# Should show drwxrwxr-x for each directory
```

Create Sample Files to Test

```
# Create a simple C program
echo '#include <stdio.h>
int main() {
    printf("Hello Embedded Linux!\n");
    return 0;
}' > ~/embedded_projects/c_programs/hello.c

# Create a shell script
echo '#!/bin/bash
echo "This is an embedded Linux setup script"
echo "Current user: $USER"
echo "Hostname: $(hostname)"
' > ~/embedded_projects/linux_scripts/setup.sh

# Make script executable
chmod +x ~/embedded_projects/linux_scripts/setup.sh

# Create a README for projects
echo '# Embedded Projects Directory
This directory contains all embedded projects.

## Structure
- c_programs/: C programming exercises
- linux_scripts/: Shell scripts for automation
- projects/: Main embedded projects
' > ~/embedded_projects/projects/README.md
```

2.4 Essential Linux Commands for Day 1

Command	Syntax	Example	Description
pwd	pwd	pwd	Print Working Directory
ls	ls [options] [path]	ls -la /home	List files/directories
cd	cd [path]	cd ~/embedded_projects	Change Directory
mkdir	mkdir [options] [dir]	mkdir -p a/b/c	Make Directory
rm	rm [options] [file]	rm -rf temp/	Remove files/dirs
cp	cp [src] [dest]	cp file1 file2	Copy files
mv	mv [src] [dest]	mv old new	Move/Rename
cat	cat [file]	cat /etc/os-release	View file content
echo	echo [text]	echo "Hello"	Print text
chmod	chmod [mode] [file]	chmod +x script.sh	Change permissions
man	man [command]	man ls	Manual pages
whoami	whoami	whoami	Current username
hostname	hostname	hostname	System hostname
uname	uname -a	uname -a	System information
df	df -h	df -h	Disk space
free	free -h	free -h	Memory usage
top/htop	top	top	Process monitoring

Practice Exercises


```
# 1. Navigate to embedded_projects
cd ~/embedded_projects

# 2. List all contents
ls -la

# 3. Go to c_programs
cd c_programs

# 4. Check file content
cat hello.c

# 5. Compile the C program
gcc hello.c -o hello

# 6. Run it
./hello
# Output: Hello Embedded Linux!

# 7. Go back to linux_scripts
cd ../linux_scripts

# 8. Run the script
./setup.sh

# 9. Check system info
uname -a

# 10. Check kernel version
uname -r
```

Part 3: Revision Material — Complete

3.1 Linux Filesystem Hierarchy — Must Know

```
/ (Root)
├── bin/           # Essential user commands (ls, cp, mv)
├── boot/          # Boot loader files (kernel, initrd)
├── dev/           # Device files (hardware interface)
├── etc/           # System configuration files
├── home/          # User home directories
│   └── embed/     # Your home directory
├── lib/           # Essential shared libraries
├── media/         # Removable media mount points
├── mnt/           # Temporary mount points
├── opt/           # Add-on software packages
├── proc/          # Virtual filesystem (process info)
├── root/          # Root user's home
├── run/           # Runtime data (temporary files)
├── sbin/          # System administration binaries
├── srv/           # Service data (web/ftp servers)
├── sys/           # Virtual filesystem (kernel parameters)
├── tmp/           # Temporary files
├── usr/           # User programs and data
│   ├── bin/       # User binaries
│   ├── lib/       # Libraries
│   ├── local/     # Locally installed software
│   └── share/     # Architecture-independent data
└── var/          # Variable data (logs, caches)
```

Important Directories for Embedded

- **/proc** - Read kernel and process information
- **/sys** - Interact with kernel and devices
- **/dev** - Access hardware devices
- **/etc** - Configuration files
- **/var/log** - System and application logs

3.2 Day 1 Quiz — Test Yourself

Question 1: What is the difference between Kernel Space and User Space?

Kernel Space:

- Runs at highest privilege level (Ring 0)
- Has direct hardware access
- Runs the core OS code
- A crash causes system failure

User Space:

- Runs at lower privilege level (Ring 3)
- Uses System Calls to access hardware
- Runs applications
- A crash only kills that application

Question 2: List 5 reasons Linux is preferred for embedded systems.

1. Open source - no licensing costs
2. Supports many CPU architectures (ARM, x86, RISC-V)
3. Highly customizable (can be stripped to 1-2MB)
4. Rich ecosystem of tools and libraries
5. Strong networking stack for IoT applications

Question 3: What command creates the directory structure /home/user/projects/src recursively?

```
mkdir -p /home/user/projects/src
```

Question 4: What is a System Call? Give 3 examples.

A system call is the interface between user space and kernel space. Examples:

- `open()` - Open a file
- `read()` - Read from a file

- write() - Write to a file
- fork() - Create a process
- exec() - Execute a program

Question 5: What do these directories contain?

- /proc - Process and kernel information (virtual)
- /sys - Kernel and device parameters (virtual)
- /dev - Device files
- /etc - System configuration
- /boot - Boot loader and kernel files

3.3 Day 1 Practical Assignment

Create and run these scripts:

Script 1: System Info Script

```
# File: ~/embedded_projects/linux_scripts/sys_info.sh
#!/bin/bash

echo "=== Embedded System Information ==="
echo "Date: $(date)"
echo "Hostname: $(hostname)"
echo "Username: $(whoami)"
echo "Kernel: $(uname -r)"
echo "Architecture: $(uname -m)"
echo "CPU Cores: $(nproc)"
echo "Memory: $(free -h | grep Mem | awk '{print $2}')"
echo "Disk Space: $(df -h / | awk 'NR==2 {print $4}')"
echo "====="
```

Script 2: Project Setup Script

```
# File: ~/embedded_projects/linux_scripts/setup_project.sh
#!/bin/bash

PROJECT_NAME=$1

if [ -z "$PROJECT_NAME" ]; then
    echo "Usage: ./setup_project.sh <project_name>"
    exit 1
fi

cd ~/embedded_projects/projects
mkdir -p $PROJECT_NAME/{src,include,build,doc}
touch $PROJECT_NAME/src/main.c
touch $PROJECT_NAME/include/config.h
touch $PROJECT_NAME/README.md

echo "# $PROJECT_NAME" > $PROJECT_NAME/README.md
echo "Created project: $PROJECT_NAME"
tree $PROJECT_NAME
```

3.4 Common Error Fixes

Error	Cause	Solution
Permission denied	No execute permission	chmod +x filename
command not found	Program not installed	sudo apt install program
cannot create directory	Missing parent directory	Use mkdir -p
No such file or directory	Wrong path	Check spelling, use pwd, ls
Unable to locate package	Wrong package name	apt search package

3.5 Day 1 Summary Notes (Quick Revision)

- ✔ Linux Kernel = Core of the operating system
- ✔ Kernel Space = Ring 0, direct hardware access
- ✔ User Space = Ring 3, applications run here

- ✔ System Calls = Bridge between User and Kernel space
- ✔ Linux is open-source, modular, supports many CPUs
- ✔ 90%+ of embedded Linux runs on ARM
- ✔ VirtualBox = Full VM experience
- ✔ WSL2 = Lightweight Linux on Windows
- ✔ Linux filesystem follows FHS standard
- ✔ /proc and /sys are virtual filesystems
- ✔ Essential commands: ls, cd, mkdir, cp, mv, rm, chmod
- ✔ Build tools: gcc, make, cmake
- ✔ Cross-compilation needed for embedded development

Day 1 Completion Checklist

Task	Completed?
Read Linux history notes	☐
Understood kernel architecture	☐
Know difference: Kernel vs User Space	☐
Can list 5 Linux features for embedded	☐
Installed Ubuntu (VirtualBox or WSL2)	☐
System updated and tools installed	☐
Created ~/embedded_projects/ structure	☐
Created c_programs/hello.c and compiled it	☐
Created linux_scripts/setup.sh	☐
Can navigate Linux filesystem	☐
Can use ls, cd, mkdir, cp, mv, rm	☐
Wrote 1-page summary on "Why Linux for embedded?"	☐
Completed the practice scripts	☐